```python
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.utils.multiclass import unique_labels
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
```

```python
# Carga de datos, interpretando ? como nulo
df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data',
  names=['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'],
  na_values='?'
)
```

```python
# Eliminar filas con valores nulos
df.dropna(inplace=True)
```

```python
# Convertir a problema de clasificación binaria. Atributo target es num, y se distinguen dos casos:
# num=0 (negativo) y num=1 (positivo)

for ind, row in df.iterrows():
  if row['num'] > 0:
    df.at[ind, 'num'] = 1
```

```python
# x: eliminar target y atributos categóricos. Utilizar los dicotómicos sex, fbs y exang como atributos predictores.
x = df.drop(['num', 'cp', 'restecg', 'slope', 'thal'], axis=1)
```

```python
# y: target
y = df['num']
```

```python
# Normalización de atributos predictores (x)
x = (x-np.min(x))/(np.max(x)-np.min(x))
```

```python
RANDOM_SEED = 42 # !!! Para poder duplicar el estudio. Para obtener los mismos resultados en cualquier ejecucion ***
```

```python
x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8, random_state=RANDOM_SEED)
```

```python
# Entrenamiento de un clasificador k-NN

k = 11

clf = KNeighborsClassifier(n_neighbors = k)
clf.fit(x_train, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier(n_neighbors=11)
```

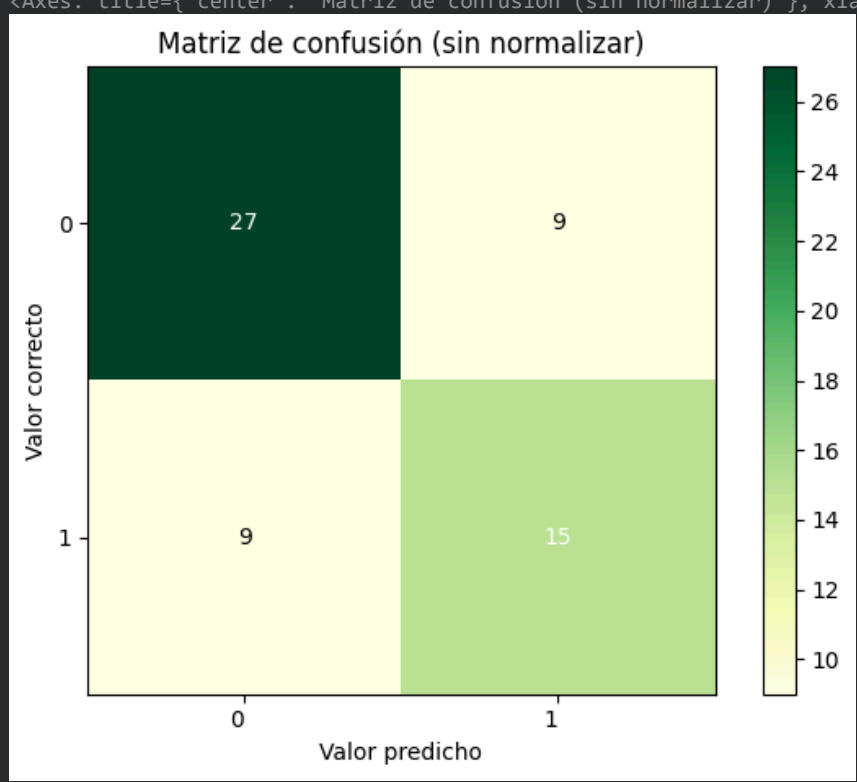
```python
# Validación

y_pred = clf.predict(x_test)
```

```python
# y_true: valores correctos
# y_pred: valores predichos
# normalize: los mismos valores que para confusion_matrix
#   ('true': valor correcto, 'pred': valor predicho, 'all': todos los valores, None: sin normalizar)
# title: título
# cmap: color map

def plot_confusion_matrix(y_true, y_pred, normalize=None,
    title=None, cmap=plt.cm.Blues, rotation=0):

  # Imprimir en forma de texto matriz de confusión y métricas

  if not title:
    if normalize == 'true':
      title = 'Matriz de confusión (normalizada por valor correcto)'
    elif normalize == 'pred':
      title = 'Matriz de confusión (normalizada por valor predicho)'
    elif normalize == 'all':
      title = 'Matriz de confusión (normalizada por totales)'
    else:
      title = 'Matriz de confusión (sin normalizar)'

  # Calcular matriz de confusión
  cm = confusion_matrix(y_true, y_pred, normalize=normalize)

  # Utilizar solo las etiquetas que aparecen en los datos
  classes = unique_labels(y_true, y_pred)

  fig, ax = plt.subplots()
  im = ax.imshow(cm, interpolation='nearest', cmap=cmap)
  ax.figure.colorbar(im, ax=ax)

  # Mostrar todos los ticks y etiquetarlos con los entradas de la lista
  ax.set(xticks=np.arange(cm.shape[1]), yticks=np.arange(cm.shape[0]),
    xticklabels=classes, yticklabels=classes,
    title=title, ylabel='Valor correcto', xlabel='Valor predicho')

  # Rotar las etiquetas para ticks y establecer su alineación
```

```
  plt.setp(ax.get_xticklabels(), rotation=rotation, ha="right",
    rotation_mode="anchor")

  # Iterar sobre los datos de la matriz para crear anotaciones de texto
  fmt = '.2f' if normalize else 'd'
  thresh = cm.max() / 2.
  for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
      ax.text(j, i, format(cm[i, j], fmt),
      ha="center", va="center",
      color="white" if cm[i, j] > thresh else "black")
  fig.tight_layout()
  return ax
```

```
plot_confusion_matrix(y_test, y_pred, cmap='YlGn', rotation=0)
```

```
<Axes: title={'center': 'Matriz de confusión (sin normalizar)'}, xlabel='Valor predicho', ylabel='Valor correcto'>
```




```
acc_rate = accuracy_score(y_test, y_pred)

print("Tasa de acierto en dataset de validación: ", acc_rate)

y_pred = clf.predict(x_train)
acc_rate = accuracy_score(y_train, y_pred)

print("Tasa de acierto en dataset de entrenamiento: ", acc_rate)
```

```
Tasa de acierto en dataset de validación:  0.7
Tasa de acierto en dataset de entrenamiento:  0.7088607594936709
```

```
y_pred = clf.predict(x_test)
```

```
print("precisión: ", precision_score(y_test, y_pred))
print("sensibilidad (recall): ", recall_score(y_test, y_pred))
print("f1-score: ", f1_score(y_test, y_pred))
```

```
precisión:  0.625
sensibilidad (recall):  0.625
f1-score:  0.625
```

Empieza a programar o a crear código con IA.

```
print("Métricas para dataset de validación: \n"
    +classification_report(y_test, y_pred))
```

```
Métricas para dataset de validación:
              precision    recall  f1-score   support

           0       0.75      0.75      0.75        36
           1       0.62      0.62      0.62        24

    accuracy                           0.70        60
   macro avg       0.69      0.69      0.69        60
weighted avg       0.70      0.70      0.70        60
```

```
cm = confusion_matrix(y_test, y_pred, labels=clf.classes_)
```

```
print('TP: ', cm[1][1])
print('TN: ', cm[0][0])
print('FP: ', cm[0][1])
print('FN: ', cm[1][0])
```

```
TP:  15
TN:  27
FP:  9
FN:  9
```

Actividad 3.2: Calculo de metricas.

Modificando el ejemplo, para que calcule todas las metricas vistas hasta ahora a partir de las cuetro posiciones del array cm.

Relacionalas con los datosmostrados al final del programa de ejemplo anterior

¿Hay alguna métrica no explicada en la intersección de las columnas “precision” y “recall” y las filas “0” y “1”? Explícala.

- En la intersección de estas como tal serían los valores de dichas columnas.
- Lo que hay es una columna "support", que hace referencia al numero de objetos unicos usados en el calculo.

```
# Asignar variables para cada valor.
TP = cm[1][1]
TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]

# -- CALCULO DE MEDIDAS --
# Precisión - TP/(TP+FP)
precision = TP/(TP+FP)
# Sensibilidad - TP/(TP+FN)
sensibilidad = TP/(TP+FN)
# Especifidad - TN/(TN+FP)
especifidad = TN/(TN+FP)
# Exactitud - TP + TN / (TP+FP+TN+FN)
exactitud = (TP+TN)/(TP+FP+TN+FN)
# F1 Score - Media armonica 2*TP/2*TP+FP+FN
f1score = 2*TP/(2*TP+FP+FN)

print(f'Precisión: {precision}\n Sensibilidad: {sensibilidad}\n Especifidad: {especifidad}')
print(f'F1 score: {f1score}')
```

```
Precisión: 0.625
 Sensibilidad: 0.625
 Especifidad: 0.75
f1 score: 0.625
```

La relación entre estas medidas y las funciones de sklearn son:

- Precision: `precision_score(y_test, y_pred)`
- Sensibilidad: `recall_score(y_test, y_pred)`
- F1score: `f1_score(y_test, y_pred)`